

Тема вебинара

AI-Architect

FinOps: архитектура управляемая стоимостью



Игорь Стурейко

Руководитель курсов: Reinforcement Learning, ML Professional, ML Basic, MLOps, FinML

Teamlead, архитектор ИИ систем,
Физический факультет МГУ, PhD теоретическая физика

Опыт:

Более 20 лет занимался прикладной математикой и мат моделированием
(Data Scientist) (Python, C++)

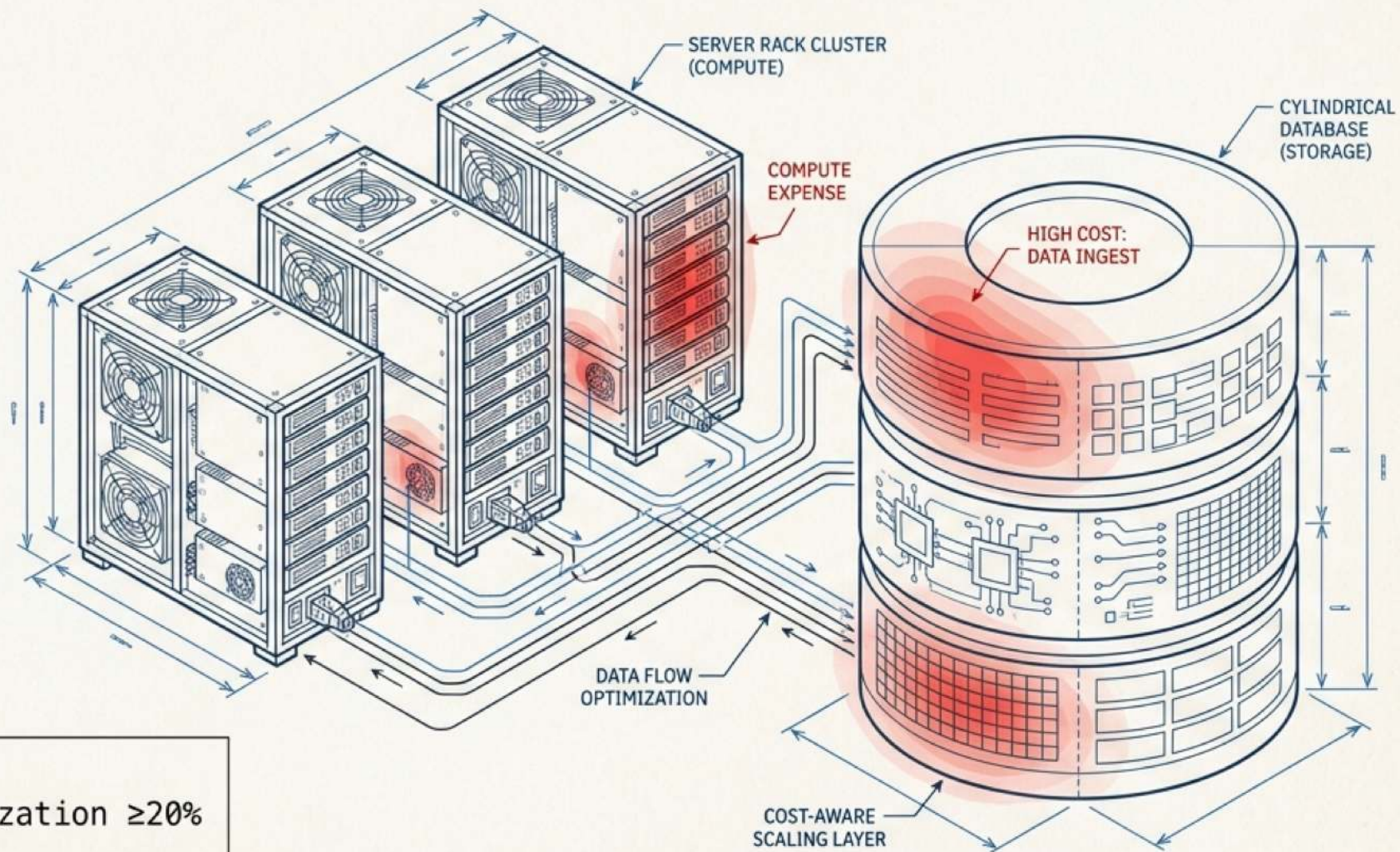
[@stureiko](#) (TG)

LinkedIn: [igor-stureiko](#)

[@rl_fintech](#) (Мой канал о моделях в бизнесе)

FinOps: архитектура, управляемая стоимостью

Как принимать архитектурные решения с учётом стоимости AI-систем

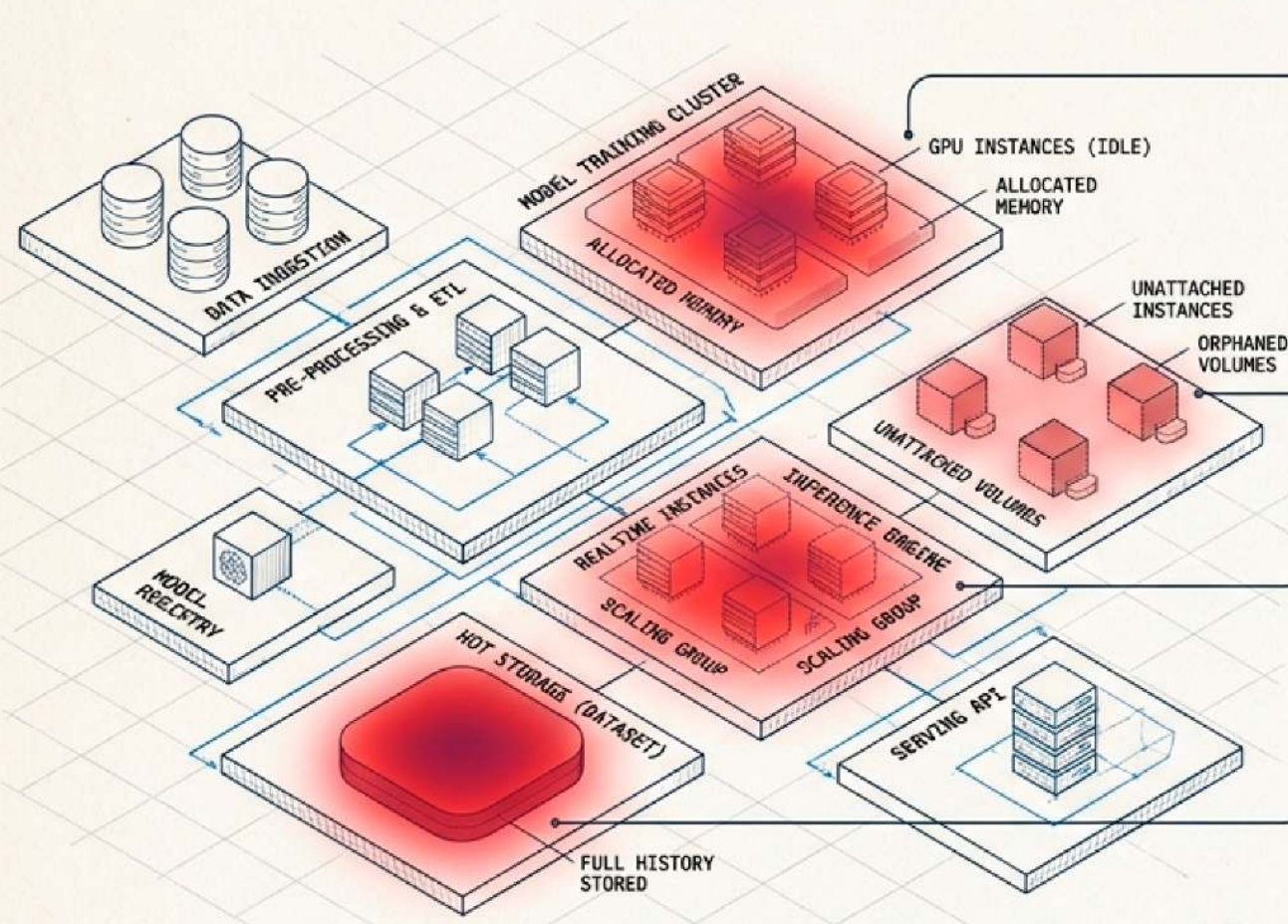


ROLE: AI Architect

TARGET_IMPACT: Cost Optimization $\geq 20\%$

STATUS: Active

Почему cloud bill выходит из-под контроля



Утечка 1: Overprovisioning

Ресурсы выделены «с запасом» и простаивают. (AI специфика: GPU ждут новых training-запусков).



Утечка 2: Zombie-ресурсы

Забутые инстансы продолжают тарифицироваться. Нет alerts и lifecycle.



Утечка 3: Дорогие Default-настройки

Хранение всего исторического датасета в Hot Storage. Inference всегда работает в realtime, даже там, где достаточно batch.



Cloud по умолчанию оптимизирован под скорость развёртывания, а не под стоимость.
Самая дорогая архитектура — та, которую потом пришлось переделывать.

Сдвиг парадигмы: FinOps — это инженерия

INIT FINOPS PROTOCOLS: [1] Visibility [2] Accountability [3] Optimization [4] Iteration

До FinOps

- Фиксированный бюджет (Accounting)
- Cost = проблема (Punishment)
- Централизованный контроль (Top-down)
- Cost-review раз в квартал (Post-mortem)

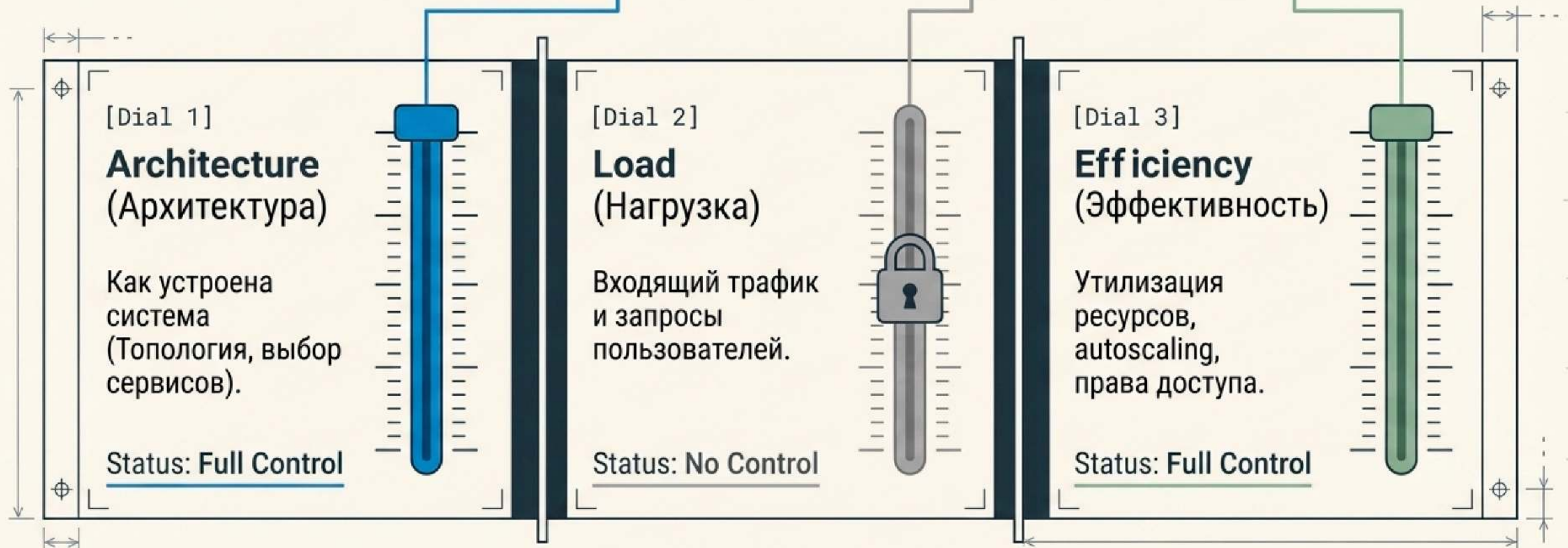
С FinOps

- Динамическая оптимизация (Engineering)
- Cost = системная метрика (Like latency or error rate)
- Распределённая ответственность (Teams own their cost)
- Cost-review на каждое архитектурное решение (Real-time)

FinOps = DevOps + Finance + Architecture.
НЕ бухгалтерия.

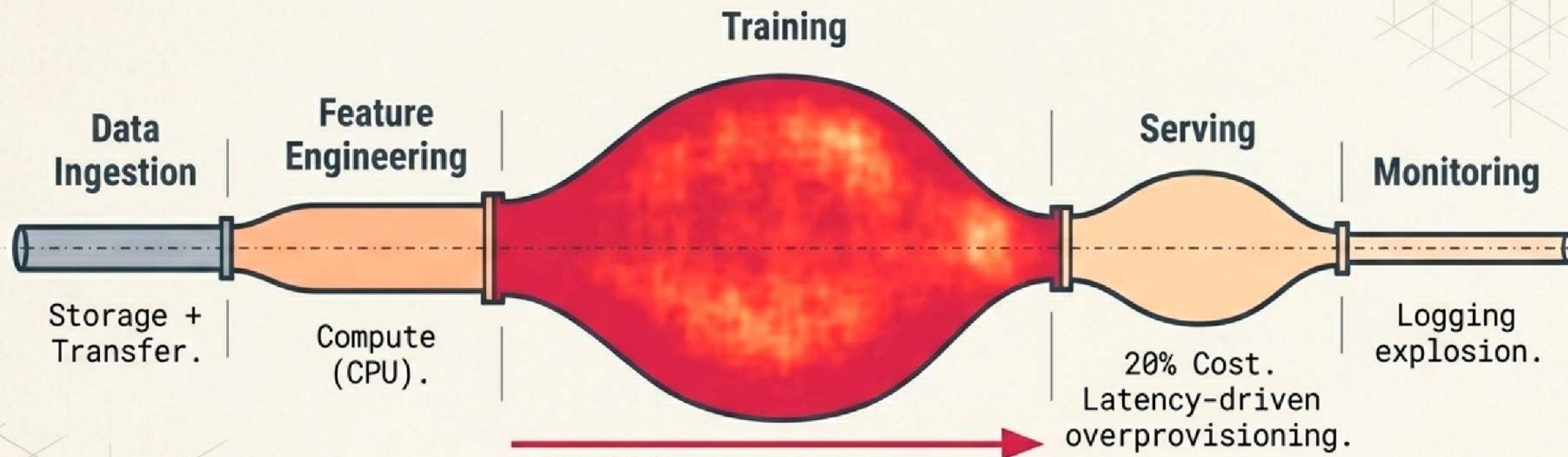
Инженерное уравнение стоимости

$$Cost = f(\textit{Architecture}, \textit{Load}, \textit{Efficiency})$$



Архитектор управляет первым и третьим. Именно здесь принимаются 80% решений о финальной стоимости системы.

Тепловая карта затрат AI/ML Pipeline



**GPU \$\$\$ —
Основная статья
расходов.**

Training забирает до 70% бюджета,
Serving — 20%. Самые дорогие решения
здесь часто принимаются «по умолчанию»
(например, постоянный Retrain).

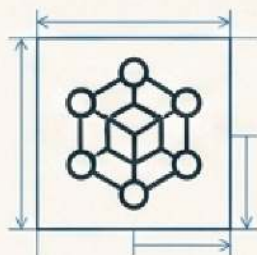
Архитектор как Cost Owner: Рычаги управления

Domain Name	Default 	FinOps Decision 
Domain 1: Compute	GPU 24/7, Realtime inference everywhere, Fixed instances. 	CPU vs GPU (только для параллелизма), Batch vs Realtime, Autoscaling (фикс = переплата 50–70% в low-traffic). 
Domain 2: Storage	Всё хранится в S3 Standard без очистки.	Настройка Lifecycle policies. Hot (Standard) → Warm (IA, -50%) → Cold (Glacier, -80–95%). Сравнение стоимости операций Object vs DB.
Domain 3: Network	Данные гоняются между регионами.	Data locality (compute и данные в одной AZ). Минимизация Egress-трафика (самый дорогой: ~\$0.08/GB).

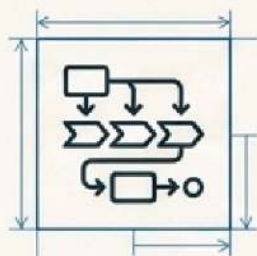
Базовый инструментарий и телеметрия



Billing: AWS Cost Explorer / GCP Billing (визуализация).



K8s Metrics: Prometheus (утилизация подов/нод) + Kubecost (cost per namespace/pod).



ML Ops: MLflow (Cost per experiment – связь эксперимента и счёта).

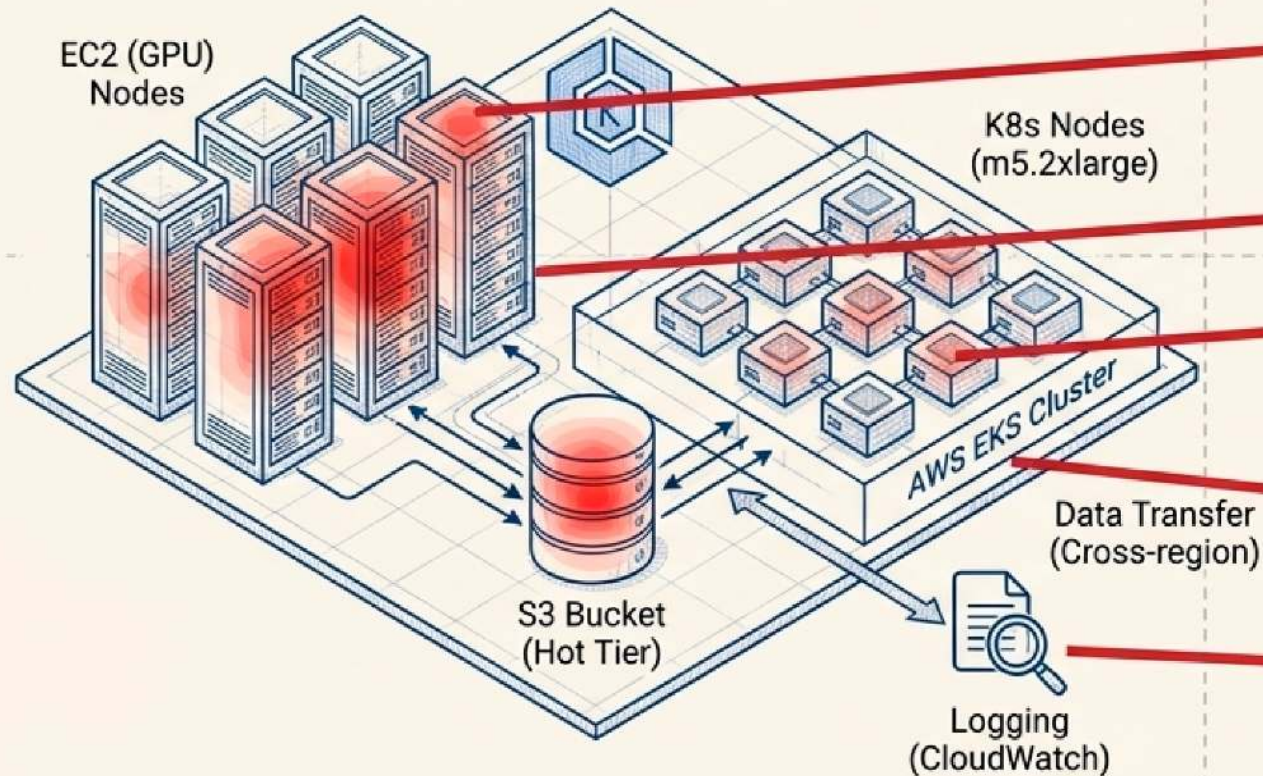
CRITICAL ALERT

ПРАВИЛО:
Нет тегов → Нет FinOps

Tagging (project / team / env / cost-center) – фундамент. Без разметки ресурсов невозможна accountability и rightsizing.

Практика: Автопсия системы «AS-IS»

Context: E-commerce рекомендации. Daily training (вся история), Realtime inference (~500 RPS пик).
Развёрнуто: AWS EKS + GPU-ноды + S3.

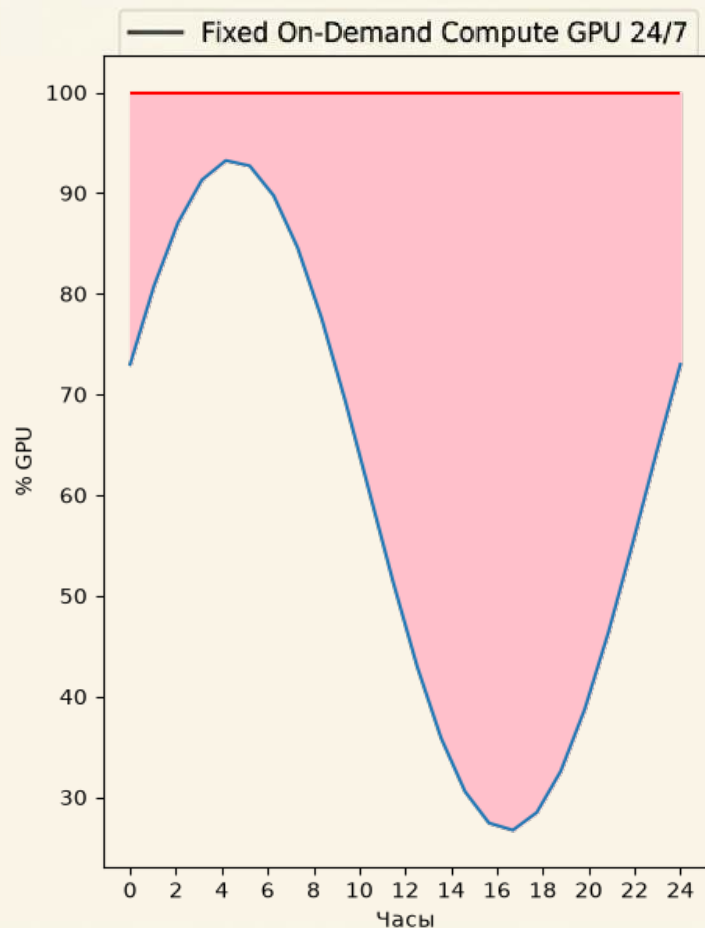


Счет / Invoice Ledger - AS-IS Cost

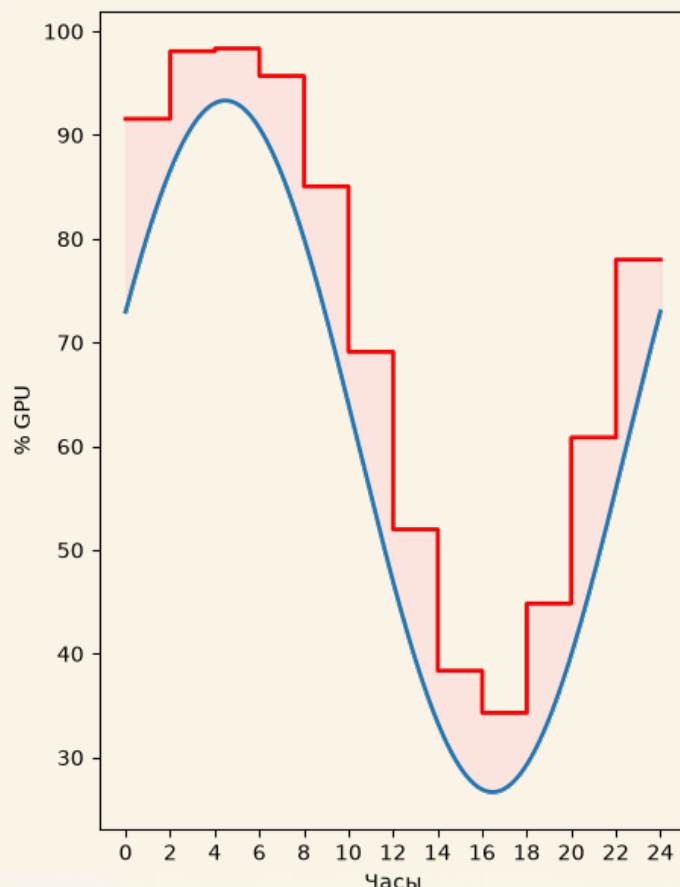
EC2 (GPU): \$12,000 (52% of budget) – [4x p3.8xlarge 24/7, 15% util]
S3: \$3,500 – [40TB Hot Tier]
K8s Nodes: \$4,000 – [8 fixed m5.2xlarge]
Data Transfer: \$2,000 – [Cross-region]
Logging: \$1,500 – [CloudWatch DEBUG]

Итого: \$23,000/мес (\$276,000/год). Архитектура намеренно плохая. **Цель:** Снизить cost $\geq 20\%$.

Операция 1: Укрощение GPU и Training Pipeline



AS-IS (Текущее состояние)



TO-BE (Целевое состояние)

Проблема: 4x p3.8xlarge молотят 24/7 с утилизацией 15%. Daily retrain при реальном drift каждые 5-7 дней.

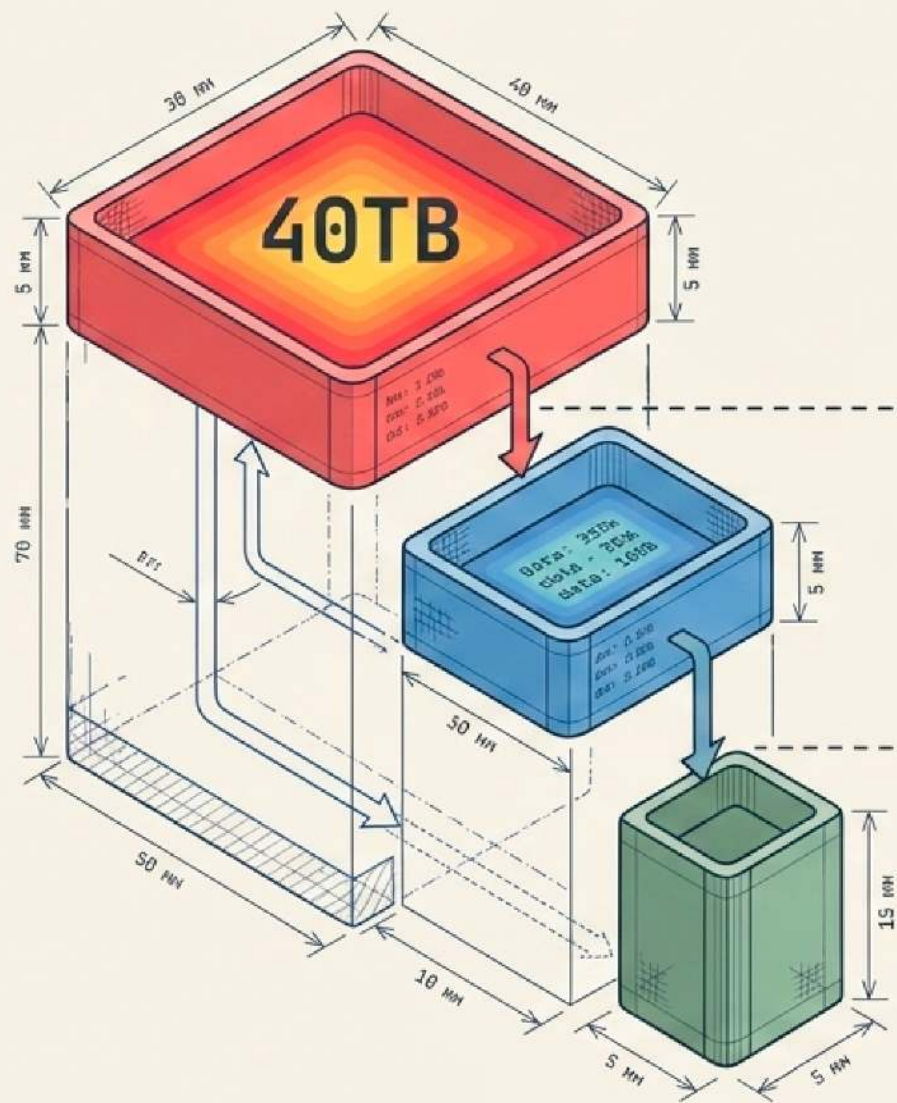
Решение А (Инфраструктура): Перевод training на Spot Instances (checkpointing каждые 10 мин) + Job Scheduling только в окно низкого трафика (00:00–06:00).

Решение Б (Алгоритмика): Переход на Weekly training + триггер по drift-метрике. Incremental training вместо full retrain.

Экономия: \$7,200–\$8,500/мес (- 60–70% от GPU бюджета).

Срок: 2–3 недели.

Операция 2: Водопад данных (S3 Lifecycle)



Hot Tier (S3 Standard):
Feature store
(горячее чтение).

>30 дней

Warm Tier (S3 IA):
Данные старше 30 дней.

>90 дней

Cold Tier (Glacier Deep Archive):
Raw logs, старые эксперименты старше 90 дней.

Проблема: 40TB исторических данных лежат в самом дорогом S3 Standard. Реально читается $\leq 5TB$.

Решение: Автоматическая Lifecycle Policy.

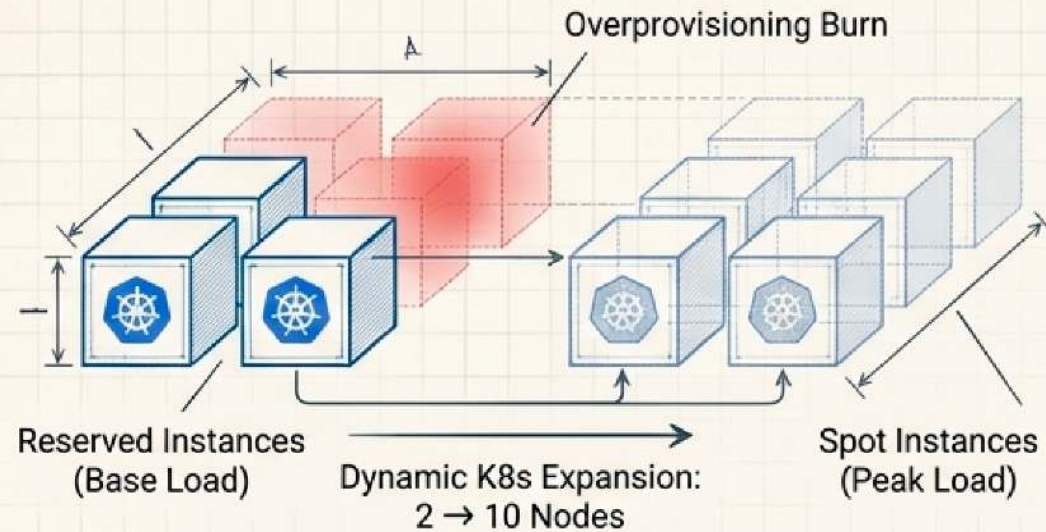
Экономия: \$2,450-\$2,800/мес (-70-80% storage cost).

Срок внедрения: 2-3 дня.
(Самый быстрый эффект).

Операция 3: Эластичный Serving и Телеметрия

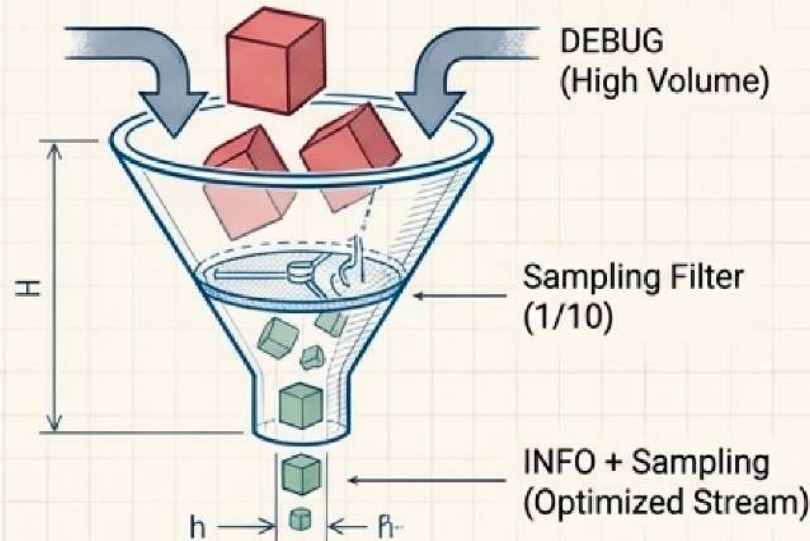
K8s Autoscaling

- **Диагностика:** 8 нод фикс. Ночью утилизация падает до 10%.
- **Решение:** Внедрение HPA по RPS/CPU для inference-подов + Cluster Autoscaler (min=2, max=10). Reserved instances для базовой нагрузки, Spot для пиков.
- **Экономия:** \$1,500-\$2,000/мес.

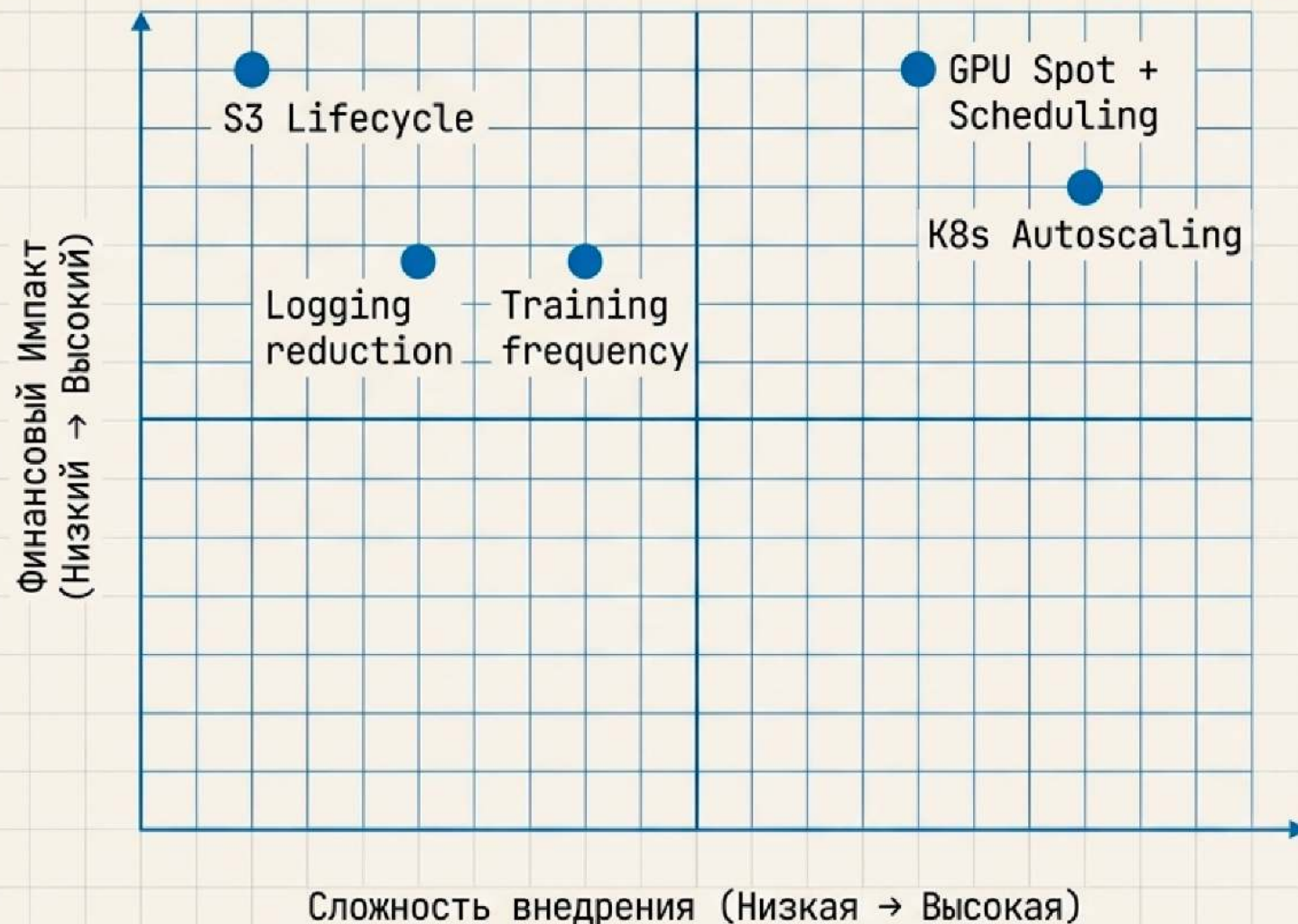


Оптимизация Логирования

- **Диагностика:** DEBUG уровень на проде, access-логи без сэмплинга.
- **Решение:** Перевод на INFO. Sampling 1/10 для healthchecks. Горячие логи 7 дней → затем в Glacier.
- **Экономия:** \$900-\$1,200/мес.



Матрица внедрения: Импакт vs Сложность



Квадрант 1 (Быстрые победы):

- S3 Lifecycle: Огромный импакт, 3 дня работы.
- Logging reduction: Средний импакт, 3 дня работы.
- Training frequency: Средний импакт, 1 неделя.

Квадрант 2 (Ключевые проекты):

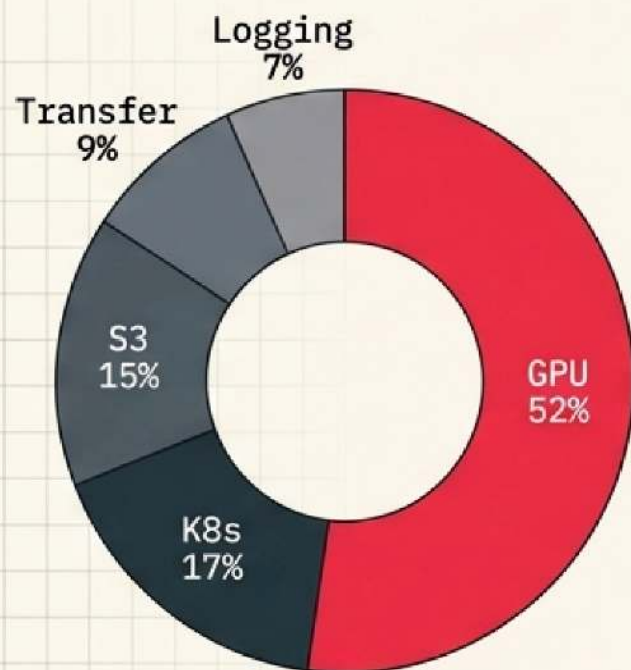
- GPU Spot + Scheduling: Максимальный импакт, 2-3 недели.
- K8s Autoscaling: Высокий импакт, 2 недели.

Takeaway: Закон Парето работает. 2 правильные оптимизации (S3 + GPU) дают 75% общей экономии.

Архитектура «ТО-ВЕ»: Синтез и Результат

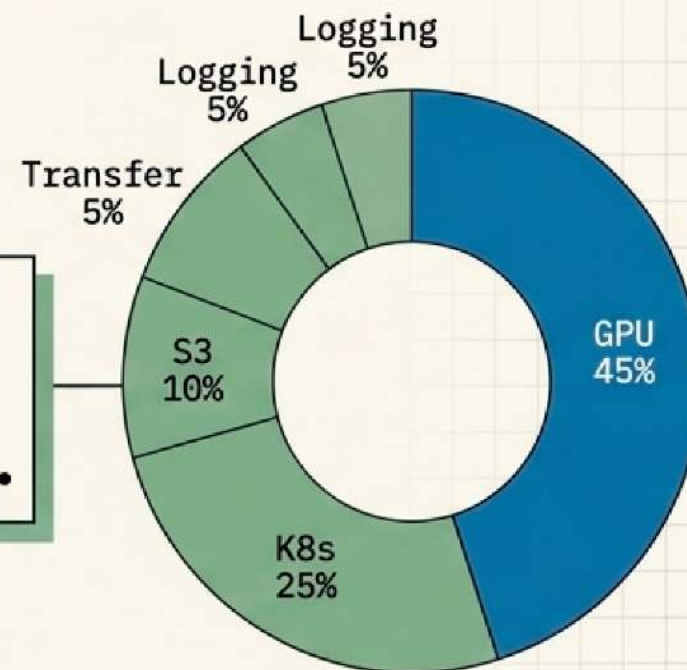
ДО (AS-IS): \$23,000/мес.

ПОСЛЕ (TO-BE): \$10,000/мес.



GPU 52%, K8s 17%, S3 15%,
Transfer 9%, Logging 7%.

Экономия:
\$12,950/мес
(≈ \$155,400/год) .



GPU 45%, K8s 25%, S3 10%,
Transfer 5%, Logging 5%.

Снижение затрат на 56% БЕЗ ухудшения качества сервиса.
Высвобожденный бюджет эквивалентен двум ставкам Senior ML-инженеров.

Золотое правило AI-Архитектора

FinOps — это не про экономию. Это про управляемость системы через стоимость. Стоимость — такой же параметр, как latency или reliability.

1

Сколько это стоит?

(Требуйте конкретные \$/мес, а не абстрактные оценки).

2

Можно ли архитектурно дешевле?

(Есть ли Spot, Batch, Cold Storage альтернативы?).

3

Что произойдет при росте нагрузки x3?

(Sensitivity analysis – сломается ли бюджет при скейлинге?).

$\text{Cost} = f(\text{Architecture}, \text{Load}, \text{Efficiency})$. Вы управляете архитектурой.

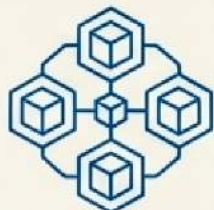
Базовый инструментарий FinOps для AI-Архитектора

Экосистема оценки, телеметрии и контроля инфраструктуры



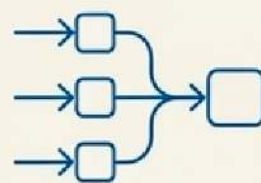
AWS Cost Explorer / Billing

Визуализация общих затрат на облако, анализ трендов и выявление аномалий в счетах.



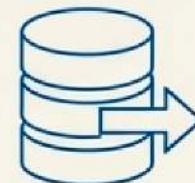
Prometheus + KubeCost

Метрики K8s. Телеметрия утилизации подов/нод и расчет точной стоимости (cost per namespace/pod).



MLflow

ML-эксперименты. Прямая привязка потребляемых вычислительных мощностей к конкретным запускам (Cost per experiment).



S3 Lifecycle Policies

Управление хранением. Автоматизация перевода данных по тирам (Hot → Warm → Cold) для радикального снижения затрат.



КРИТИЧЕСКОЕ УСЛОВИЕ: Тегирование (Tagging)

Правило: Нет тегов → Нет FinOps. Без строгой разметки ресурсов (project / team / env / cost-center) невозможны accountability и rightsizing ни в одном из инструментов выше.

Сдвиг парадигмы: Prevention over Correction

[(Стратегия «Shift Left» в FinOps)]

Prevention (Shift Left)

Correction (Post-factum)

[Phase 1] Design / Архитектура

Brainboard

Класс: Architecture-as-Code

- Визуальное проектирование облачной инфраструктуры.
- Автоматическая генерация Terraform-кода.

FinOps Impact: Оценка стоимости решений в реальном времени прямо на чертеже — до написания первой строчки кода.



```
endless Infrastructure = {
  class = {
    name: "aws-ec2"
    resource: "aws-ec2"
    resource: "aws-ec2"
    resource: "aws-ec2"
  }
  class = {
    type: "code"
  }
}
```

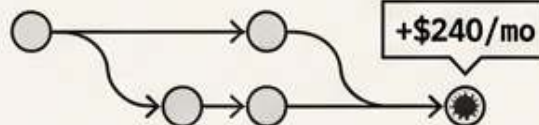
[Phase 2] Build / CI/CD

Infracost

Класс: Cost-as-Code

- Анализ изменений в Terraform-планах на этапе Pull Requests.
- Нативная интеграция прямо в CI/CD пайплайн.

FinOps Impact: Показывает дельту изменения счета за облако до развертывания ресурсов (блокировка дорогих коммитов).



[Phase 3] Run / Production

AWS Cost Explorer / GCP Billing / Kubecost

Статус: Контроль «по факту»
(Post-mortem)

Ресурсы уже запущены, деньги уже тратятся.

Самая дорогая архитектура
— та, которую потом
пришлось переделывать.

Main Takeaway: Переход от контроля счетов в конце месяца к инженерному управлению стоимостью на этапе чертежа и кода. Стоимость — это системная метрика, такая же как latency.

Приходите на следующие вебинары



Игорь Стурейко

Руководитель курсов: Reinforcement Learning, ML Professional, ML Basic, MLOps, FinML

Teamlead, архитектор ИИ систем,
Физический факультет МГУ, PhD теоретическая физика

Опыт:

Более 20 лет занимался прикладной математикой и мат моделированием (Data Scientist) (Python, C++)

[@stureiko](#) (TG)

LinkedIn: [igor-stureiko](#)

[@rl_fintech](#) (Мой канал о моделях в бизнесе)